

AIOps Assignment 2

Infrastructure & Containerization
Krishanthh Mohan : DA24B044

Q1: Single-Stage vs. Multi-Stage Docker

The single-stage image used python:3.12, copied the entire project directory, and had a size of 584 MB. Both /healthz and /predict were verified.

The multi-stage image used a builder stage for dependency wheels and a final python:3.12-slim runtime stage. Its size was 193 MB, giving a reduction of

$$\frac{584 - 193}{584} \times 100 \approx 67.0\%$$

The final image is smaller because not everything (the full python:3.12 base, training files, Pandas dependency, build files, and wheel cache) is retained. Only the API, trained model, and runtime dependencies are present.

Q2: Docker Compose and Redis

The API was extended with Redis caching. The input text is used as the cache key. On a miss, the model computes the prediction and stores it with a 600-second TTL; on a hit, the cached label is returned without rerunning the model. The TTL was also verified by running 11 minutes later on the same input text, a cache miss was received again, despite the cache not being manually cleared.

The Compose application contained two services: api and redis-server, where redis-server used the official redis:7-alpine image. The API accessed Redis using the Compose service name redis-server as its hostname.

Measured request times:

- Cache miss: [0.043228]
- Cache hit: [0.009770]

The second request was faster because model inference was skipped.

Docker Compose provides single-host orchestration and networking between multiple related containers, while Kubernetes operates at cluster scale and also provides scheduling, self-healing, scaling, and rolling-update mechanisms.

Q3: Kubernetes Indexed Job

Eight CSV shards were processed using an Indexed Job with:

```
completions: 8
parallelism: 4
completionMode: Indexed
```

The cluster had 2 nodes with 2 allocatable CPUs each, giving 4 CPUs total. Each pod requested 1 CPU, so at most

$$4/1 = 4$$

Pods could run simultaneously. Therefore, parallelism: 4 was used. `kubectl get pods -o wide` confirmed four concurrent pods split equally across both of the Minikube nodes.

Each pod used `JOB_COMPLETION_INDEX` to select exactly one shard. The Downward API exposed the pod and node names through `POD_NAME` and `NODE_NAME`.

Invalid-row counts obtained from pod logs were:

Shard	0	1	2	3	4	5	6	7
Invalid rows	12	10	9	8	7	6	5	4

Pod logs were used instead of a shared volume because Minikube's default storage provisioner is node-local, while the results are small and can be collected directly through the Kubernetes API.

With 3 nodes and 2 CPUs per node, the cluster would have 6 CPUs. With the same 1-CPU pod request, I can increase parallelism to 6 while keeping completions: 8.

Q4: Kubernetes Deployment

The spam API was deployed using a Deployment with 2 replicas, CPU requests/limits, a readiness probe on /healthz, and a Service.

To test self-healing, one pod was manually deleted. The deployment detected that the actual replica count had fallen below the desired count of 2 and automatically created a replacement pod.

For the rolling update, /healthz was modified to include a version string and a new image spam-api:v2 was deployed. kubectl rollout status reported a successful rollout, and kubectl rollout history showed the Deployment revisions.

A Deployment is appropriate for the API because it is a long-running serving workload that must remain available and support self-healing and rolling updates. A Job is appropriate for the shard-validation workload because it performs finite batch work and terminates after completion.